Multi-select Dropdown with Search

Problem: Build a multi-select dropdown: a search input that filters a list of options, lets the user pick several of them with the mouse or the keyboard, and shows the picks as removable chips.
Often asked at: Flipkart, PhonePe, Razorpay
Why it matters: accessibility, click-outside, keyboard
Tech stack: React (Vite, JavaScript) *(default, not specified)*
Time limit: 60 minutes *(default, not specified)*

PART A: Thought process

The general steps for any machine coding task

1. **Understand the requirements:** functional, non-functional, clarifying questions, assumptions.
2. **List the interactions:** every action and its expected result. This becomes your checklist.
3. **Design the architecture:** components and their single responsibility.
4. **Design the state:** the smallest state that can describe the UI; derive the rest.
5. **Design data and contracts:** data shapes, component props, API payloads.
6. **Trace the data flow** of one key interaction end to end.
7. **Find the hard parts** early and decide how to solve them.
8. **Build in layers:** simplest working version first, then add one working layer at a time.
9. **Cover states and edge cases.**
10. **Wrap up:** trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, clarifying questions, assumptions	5 min
Interactions, architecture, state, contracts	8 min
Pure utils (filter, toggle, highlight split)	5 min
Input + filtered list + click to toggle + chips	12 min
Open / close, click outside, Escape, Tab	8 min
Keyboard navigation, active option, scroll into view	10 min
ARIA combobox wiring, live region, Select all / Clear all	8 min
Wrap-up and trade-offs	4 min

Tip: think out loud

Say what you are deciding and why: *"I'll store the selection as an array of ids, not objects, so it survives the options list changing"*, *"I'll keep focus in the input the whole time and point at the active option with* `aria-activedescendant` *"*. The interviewer grades your reasoning as much as your code. If you run out of time, a clearly explained gap beats a silent half-feature.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A text input filters the options as you type (case-insensitive, "contains").
- Clicking an option toggles it; the list stays open so you can pick more.
- Selected options show as chips inside the control; each chip has a remove (×) button.
- Keyboard: ↓ / ↑ move the active option, Enter toggles it, Escape closes, Backspace on an empty input removes the last chip, Tab leaves and closes.
- Clicking outside the component closes the dropdown.
- "Select all" (the visible / filtered options) and "Clear all".
- Empty state: "No results for 'xyz'".
- Matching part of each label is highlighted.

Non-functional requirements

Area	What it means here
Performance	Filtering is O(n) per keystroke; fine for a few hundred options. Memoise derived lists.
Scalability	For thousands of options, add debounce and list virtualisation (wrap-up).
Error handling	Unknown ids in `value` are ignored, never crash the render.
Accessibility	WAI-ARIA combobox + listbox pattern, `aria-multiselectable`, `aria-activedescendant`, labelled remove buttons, live region for counts.
Usability	Focus never leaves the input while picking; active option scrolls into view.
Reusability	Controlled component: `value` + `onChange`, options passed in as props.
Extensibility	Pure utils make it easy to swap the matcher (fuzzy, server search).

Clarifying questions to ask the interviewer

1. Are the options static, or do they come from an API as the user types?
2. Roughly how many options: tens, hundreds, or thousands?
3. Should the dropdown stay open after a selection, or close each time?

4. Is there a max number of selections, or disabled options?
5. Should "Select all" select everything, or only what the search currently shows?
6. Do selected chips need to keep the order the user picked them in?

Assumptions we will build with

- Options are a static array of `{ id, label }` passed as a prop (20 skills in the demo).
- The dropdown stays open after a pick (that is the whole point of multi-select).
- No max, no disabled options (named in the wrap-up as extensions).
- "Select all" selects the **visible** (filtered) options, adding to the current selection.
- Chips keep the order of selection.
- The component is controlled: the parent owns the selected ids.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Focus or click the control	Dropdown opens with all options, first one active
Type "scr"	List filters to matching labels, "scr" highlighted, active resets to first
Click an option	Toggles it; chip appears / disappears; list stays open; focus stays in input
↓ / ↑	Moves the active option, wraps around at the ends, scrolls it into view
Home / End	Jumps to first / last option
Enter	Toggles the active option (does not submit a parent form)
↓ while closed	Opens the dropdown
Escape	Closes the dropdown and clears the query; focus stays in input
Backspace with empty input	Removes the last chip
Click × on a chip	Removes that option; focus returns to the input
Click "Select all"	Adds every visible option not already selected
Click the clear (×) button	Empties the selection
Click outside	Closes the dropdown, clears the query
Tab	Closes the dropdown, focus moves on normally
Query with no matches	Shows "No results for 'xyz'"
Screen reader	Hears "combobox, expanded", the active option and its selected state, and "N results. M selected."

3. Design the architecture

Module	Responsibility
`App`	Owns the selected ids, renders the demo and the output
`MultiSelect`	The whole widget: query, open state, active option, keyboard, ARIA
`Highlight`	Renders a label with the matched part in `<mark>`
`useClickOutside`	Calls a handler on pointerdown outside a ref, only while enabled
`utils/options.js`	Pure functions: `filterOptions`, `toggleValue`, `splitByMatch`
`data/skills.js`	Mock options with stable ids

```
App  (value = selected ids)
└── MultiSelect  (label, options, value, onChange)
    ├── control
    │   ├── chips  (one per selected id, × button each)
    │   ├── input  role="combobox"
    │   └── clear-all button
    ├── popup  (only when open)
    │   ├── actions bar  ("N results", Select all)
    │   └── listbox  role="listbox" aria-multiselectable
    │       └── option × n  → Highlight
    └── live region  ("N results. M selected.")

useClickOutside(rootRef, close, isOpen)
utils: filterOptions · toggleValue · splitByMatch
```

Start with one `MultiSelect` component. Split out `Chip` or `OptionList` only if they grow; the hook and the pure utils are split from the start because they are easy to test alone.

4. Design the state / data model

State or field	Purpose
`value` (prop, owned by parent)	Selected ids, in selection order
`isOpen`	Is the popup visible
`query`	Current search text
`activeIndex`	Index of the keyboard-active option **inside the filtered list**
`rootRef`	Boundary for click-outside
`inputRef`	Return focus to the input after chip remove / clear

State or field	Purpose
`listRef`	Find the active option element to scroll it into view

```
// Parent
const [skills, setSkills] = useState(["react"]);

// MultiSelect
const [isOpen, setIsOpen] = useState(false);
const [query, setQuery] = useState("");
const [activeIndex, setActiveIndex] = useState(0);

// Derived, never stored
const selectedSet = useMemo(() => new Set(value), [value]);
const filtered = useMemo(() => filterOptions(options, query), [options, query]);
const selectedOptions = value.map((v) => optionById.get(v)).filter(Boolean);
const activeOption = isOpen ? filtered[activeIndex] : undefined;
```

Senior note:

- `filtered`, `selectedSet`, `selectedOptions` and `activeOption` are all **derived**. Storing any of them would create two sources of truth that drift apart (e.g. a stale filtered list after options change).
- `value` belongs to the **parent** (controlled), because the parent submits it with a form. The **component** owns the UI-only state: `isOpen`, `query`, `activeIndex`.
- The selection is stored as **ids**, not option objects: comparing ids is cheap, it survives a refetch of options, and labels can repeat.
- `activeIndex` is an index into `filtered`, so every change to `query` must reset it (done in the change handler, not an effect).

5. Design the data structures and contracts

Example data

```
export const SKILLS = [
  { id: "js", label: "JavaScript" },
  { id: "ts", label: "TypeScript" },
  { id: "react", label: "React" },
  // ...
];
```

Stable `id`s are used for React keys, for the selected value, and for the DOM ids that `aria-activedescendant` points to. Labels are only for display and search.

Component contract

```
<MultiSelect
  label="Skills"                 // visible label + listbox name
  options={[{ id, label }]}      // all options
  value={["react", "ts"]}        // selected ids, in order
  onChange={(nextIds) => {}}     // called with the full new array
  placeholder="Search skills..." // shown when nothing is selected
/>
```

If options came from an API (no real API here, so this is the mock contract):

```
GET /api/skills?q=scr&limit=50
200 → { "items": [ { "id": "js", "label": "JavaScript" }, { "id": "ts", "label": "TypeScript" } ] }
```

The component would then keep a `Map` of every option it has ever seen, so chips for selected ids still render after the search results change.

6. Trace the data flow

Key interaction: the user types "script", presses ↓, then Enter.

```
keystroke "script"
   │
   ▼
handleQueryChange ─► setQuery("script"), setActiveIndex(0), open()
   │
   ▼
re-render ─► filtered = filterOptions(options, "script") = [JavaScript, TypeScript]
          ─► Highlight splits labels: "Java" + <mark>Script</mark>
          ─► live region: "2 results. 1 selected."
   │
ArrowDown ─► move(1) ─► setActiveIndex((0 + 1) % 2) = 1
   │
   ▼
re-render ─► activeOption = TypeScript
          ─► input aria-activedescendant = "<id>-opt-ts"
          ─► effect: scrollIntoView({ block: "nearest" }) on data-index=1
   │
Enter ─► preventDefault ─► toggle("ts") ─► onChange(toggleValue(value, "ts"))
   │
   ▼
parent setSkills(["react", "ts"]) ─► new value prop
   │
```

```
   ▼
re-render ─► chip "TypeScript" appears, option aria-selected="true", checkbox filled
```

7. Important technical challenges

Challenge 1: Click outside without breaking clicks inside

Problem: The dropdown must close when the user clicks anywhere else. A naive `onBlur` on the input closes the list *before* the click on an option lands (blur fires on mousedown), so the option click is lost.

Solution: Listen for `pointerdown` on `document` only while open, and close if the target is not inside the root. On options and the "Select all" button, `preventDefault` on `mousedown` so the input never loses focus.

```
useEffect(() => {
  if (!enabled) return;
  const onPointerDown = (e) => {
    if (ref.current && !ref.current.contains(e.target)) handlerRef.current(e);
  };
  document.addEventListener("pointerdown", onPointerDown);
  return () => document.removeEventListener("pointerdown", onPointerDown);
}, [ref, enabled]);

// on each option
onMouseDown={(e) => e.preventDefault()} // keep focus in the input
```

The handler is kept in a ref so the listener is not removed and re-added on every render.

Challenge 2: Keyboard navigation with focus staying in the input

Problem: If arrow keys moved real DOM focus onto options, the user could no longer type to refine the search, and screen readers would lose the combobox context.

Solution: Keep focus in the input. Track `activeIndex` in state, style that option as active, and tell assistive tech with `aria-activedescendant`. Wrap around with modulo; reset to 0 when the query changes.

```
const move = (step) => {
  if (!isOpen) return open();
  if (!filtered.length) return;
  setActiveIndex((i) => (i + step + filtered.length) % filtered.length);
};

<input
  role="combobox"
  aria-expanded={isOpen}
  aria-controls={listboxId}
```

```
  aria-activedescendant={activeOption ? optionDomId(activeOption.id) : undefined}
/>
```

Challenge 3: Keeping the active option visible

Problem: With 20 options in a 240px list, pressing ↓ ten times moves the active option below the fold; the user is "navigating blind".

Solution: After `activeIndex` changes, scroll that element into view with `block: "nearest"` (only scrolls when needed, never jumps the page).

```
useEffect(() => {
  if (!isOpen) return;
  listRef.current
    ?.querySelector(`[data-index="${activeIndex}"]`)
    ?.scrollIntoView({ block: "nearest" });
}, [activeIndex, isOpen]);
```

Challenge 4: Accessible multi-select semantics

Problem: A `div` list with click handlers is invisible to screen readers: they don't know it is a list of choices, that several can be picked, which one is active, or what is selected.

Solution: Use the ARIA 1.2 combobox pattern. `role="listbox"` with `aria-multiselectable="true"`, each item `role="option"` with `aria-selected`, chip remove buttons with `aria-label="Remove React"`, a real `<label>` for the input, and a polite live region for counts.

```
<ul role="listbox" aria-multiselectable="true" aria-label={label}>
  <li role="option" aria-selected={selected} id={optionDomId(o.id)}>…</li>
</ul>
<p className="sr-only" aria-live="polite">
  {isOpen ? `${filtered.length} results. ` : ""}{value.length} selected.
</p>
```

8. Implement in the correct order

#	Step	Working result
1	Mock data + `App` with `useState` for selected ids	Page renders, value printed as JSON
2	`MultiSelect` with input + full list always visible	Options render
3	`filterOptions` + `query` state	Typing filters the list
4	`toggleValue` + click to toggle + checkbox look	Mouse selection works
5	Chips with × remove	Selection visible and removable

#	Step	Working result
6	`isOpen` + open on focus/click, close on Escape	Popup behaves
7	`useClickOutside` + `mousedown` preventDefault on options	Clicks outside close, clicks inside keep focus
8	`activeIndex` + ↓ ↑ Home End Enter	Full keyboard selection
9	scrollIntoView effect	Active option always visible
10	Backspace removes last chip, Tab closes	Power-user keys
11	ARIA combobox / listbox / live region	Screen reader support
12	Select all (visible), Clear all, empty state, highlight	Polish

9. Cover all states and edge cases

State / edge case	Handling
Initial (closed, preset value)	Chips render from `value`, placeholder hidden
Initial (closed, empty)	Placeholder "Search skills..." shown
Open, empty query	All options, first active
Filtering	List updates per keystroke, active resets to 0
Empty results	"No results for 'xyz'"; ↓ ↑ Enter do nothing; live region says 0 results
Whitespace-only query	Trimmed: treated as empty, shows everything
Everything selected	All options checked; "Select all" adds nothing (no duplicates)
Nothing selected	Clear button hidden; Backspace on empty input does nothing
`value` has an unknown id	`filter(Boolean)` skips it: no crash, no ghost chip
Enter inside a `<form>`	`preventDefault` so it toggles instead of submitting
Escape while closed	Not prevented, so a parent modal can still close
Click option	`mousedown` preventDefault keeps focus; list stays open
Click chip ×	`stopPropagation` so the control doesn't also toggle open; focus back to input
Clicking the focused input after Escape	Control click calls `open()` (focus event won't fire again)
Mouse hover vs keyboard	Hover sets `activeIndex`, so mouse and keys never show two active rows
Long list	`max-height` + scroll; active option scrolled into view
Many chips	Control wraps onto more lines (`flex-wrap`)

State / edge case	Handling
Two dropdowns on one page	`useId` gives unique listbox / option ids

10. Final structure

```
src/
├── components/
│   └── MultiSelect/
│       ├── MultiSelect.jsx     # widget + Highlight
│       └── MultiSelect.css
├── hooks/
│   └── useClickOutside.js
├── utils/
│   └── options.js              # filterOptions, toggleValue, splitByMatch
├── data/
│   └── skills.js               # mock options
├── App.jsx
├── main.jsx
└── index.css
```

11. Full working code

`src/utils/options.js`

```
// Case-insensitive "contains" match on the label. Empty query = everything.
export function filterOptions(options, query) {
  const q = query.trim().toLowerCase();
  if (!q) return options;
  return options.filter((o) => o.label.toLowerCase().includes(q));
}

// Add the id if missing, remove it if present. Keeps selection order.
export function toggleValue(value, id) {
  return value.includes(id) ? value.filter((v) => v !== id) : [...value, id];
}

// "JavaScript", "script" -> ["Java", "Script", ""] so the match can be wrapped in <mark>.
export function splitByMatch(text, query) {
  const q = query.trim().toLowerCase();
  const start = q ? text.toLowerCase().indexOf(q) : -1;
  if (start === -1) return [text, "", ""];
```

```
  return [text.slice(0, start), text.slice(start, start + q.length), text.slice(start + q.length)];
}
```

src/hooks/useClickOutside.js

```
import { useEffect, useRef } from "react";

// Calls handler on a pointerdown outside ref. Only listens while enabled.
export default function useClickOutside(ref, handler, enabled = true) {
  const handlerRef = useRef(handler);

  useEffect(() => {
    handlerRef.current = handler; // latest handler without re-subscribing
  }, [handler]);

  useEffect(() => {
    if (!enabled) return;
    const onPointerDown = (e) => {
      if (ref.current && !ref.current.contains(e.target)) handlerRef.current(e);
    };
    document.addEventListener("pointerdown", onPointerDown);
    return () => document.removeEventListener("pointerdown", onPointerDown);
  }, [ref, enabled]);
}
```

src/data/skills.js

```
// Stable ids: selection is stored by id, labels can change or repeat.
export const SKILLS = [
  { id: "js", label: "JavaScript" },
  { id: "ts", label: "TypeScript" },
  { id: "react", label: "React" },
  { id: "next", label: "Next.js" },
  { id: "vue", label: "Vue" },
  { id: "angular", label: "Angular" },
  { id: "svelte", label: "Svelte" },
  { id: "node", label: "Node.js" },
  { id: "express", label: "Express" },
  { id: "graphql", label: "GraphQL" },
  { id: "redux", label: "Redux" },
  { id: "html", label: "HTML" },
  { id: "css", label: "CSS" },
  { id: "tailwind", label: "Tailwind CSS" },
  { id: "jest", label: "Jest" },
```

```
  { id: "rtl", label: "React Testing Library" },
  { id: "webpack", label: "Webpack" },
  { id: "vite", label: "Vite" },
  { id: "docker", label: "Docker" },
  { id: "aws", label: "AWS" },
];
```

src/components/MultiSelect/MultiSelect.jsx

```
import { useEffect, useId, useMemo, useRef, useState } from "react";
import useClickOutside from "../../hooks/useClickOutside";
import { filterOptions, splitByMatch, toggleValue } from "../../utils/options";
import "./MultiSelect.css";

function Highlight({ text, query }) {
  const [before, match, after] = splitByMatch(text, query);
  return (
    <span>
      {before}
      {match && <mark>{match}</mark>}
      {after}
    </span>
  );
}

export default function MultiSelect({ label, options, value, onChange, placeholder = "Search..." })
{
  const [isOpen, setIsOpen] = useState(false);
  const [query, setQuery] = useState("");
  const [activeIndex, setActiveIndex] = useState(0);

  const rootRef = useRef(null);
  const inputRef = useRef(null);
  const listRef = useRef(null);
  const id = useId();
  const listboxId = `${id}-listbox`;
  const optionDomId = (optId) => `${id}-opt-${optId}`;

  // Derived, never stored.
  const selectedSet = useMemo(() => new Set(value), [value]);
  const optionById = useMemo(() => new Map(options.map((o) => [o.id, o])), [options]);
  const filtered = useMemo(() => filterOptions(options, query), [options, query]);
  const selectedOptions = value.map((v) => optionById.get(v)).filter(Boolean);
  const activeOption = isOpen ? filtered[activeIndex] : undefined;
```

```
  const open = () => setIsOpen(true);
  const close = () => {
    setIsOpen(false);
    setQuery("");
    setActiveIndex(0);
  };
  useClickOutside(rootRef, close, isOpen);

  const toggle = (optId) => onChange(toggleValue(value, optId));
  const remove = (optId) => onChange(value.filter((v) => v !== optId));

  const selectAllVisible = () => {
    const missing = filtered.filter((o) => !selectedSet.has(o.id)).map((o) => o.id);
    onChange([...value, ...missing]);
  };
  const clearAll = () => {
    onChange([]);
    inputRef.current?.focus();
  };

  // Keep the keyboard-active option visible inside the scrolling list.
  useEffect(() => {
    if (!isOpen) return;
    listRef.current?.querySelector(`[data-index="${activeIndex}"]`)?.scrollIntoView({ block:
"nearest" });
  }, [activeIndex, isOpen]);

  const handleQueryChange = (e) => {
    setQuery(e.target.value);
    setActiveIndex(0); // reset in the handler, not an effect
    open();
  };

  const move = (step) => {
    if (!isOpen) return open();
    if (!filtered.length) return;
    setActiveIndex((i) => (i + step + filtered.length) % filtered.length);
  };

  const handleKeyDown = (e) => {
    switch (e.key) {
      case "ArrowDown":
        e.preventDefault();
        move(1);
```

```
      break;
    case "ArrowUp":
      e.preventDefault();
      move(-1);
      break;
    case "Home":
    case "End":
      if (!isOpen || !filtered.length) return;
      e.preventDefault();
      setActiveIndex(e.key === "Home" ? 0 : filtered.length - 1);
      break;
    case "Enter":
      if (!isOpen) return open();
      e.preventDefault(); // don't submit a surrounding form
      if (activeOption) toggle(activeOption.id);
      break;
    case "Escape":
      if (isOpen) {
        e.preventDefault();
        close();
      }
      break;
    case "Backspace":
      if (!query && value.length) remove(value[value.length - 1]);
      break;
    case "Tab":
      close();
      break;
  }
};

return (
  <div className="ms" ref={rootRef}>
    <label className="ms-label" htmlFor={`${id}-input`}>
      {label}
    </label>

    <div
      className={`ms-control ${isOpen ? "is-open" : ""}`}
      onClick={() => {
        inputRef.current?.focus();
        open(); // focus alone won't reopen an already-focused input
      }}
    >
      <ul className="ms-chips" aria-label={`Selected ${label}`}>
```

```
          {selectedOptions.map((o) => (
            <li key={o.id} className="ms-chip">
              {o.label}
              <button
                type="button"
                className="ms-chip-remove"
                aria-label={`Remove ${o.label}`}
                onClick={(e) => {
                  e.stopPropagation();
                  remove(o.id);
                  inputRef.current?.focus();
                }}
              >
                ×
              </button>
            </li>
          ))}
        </ul>

        <input
          ref={inputRef}
          id={`${id}-input`}
          className="ms-input"
          type="text"
          role="combobox"
          aria-expanded={isOpen}
          aria-controls={listboxId}
          aria-autocomplete="list"
          aria-activedescendant={activeOption ? optionDomId(activeOption.id) : undefined}
          placeholder={value.length ? "" : placeholder}
          value={query}
          onChange={handleQueryChange}
          onKeyDown={handleKeyDown}
          onFocus={open}
          autoComplete="off"
        />

        {value.length > 0 && (
          <button type="button" className="ms-clear" aria-label="Clear all" onClick={(e) => {
              e.stopPropagation();
              clearAll();
            }}
          >
            ×
          </button>
```

```
        )}
      </div>

      {isOpen && (
        <div className="ms-popup">
          {filtered.length > 0 && (
            <div className="ms-actions">
              <span>{filtered.length} results</span>
              <button type="button" onMouseDown={(e) => e.preventDefault()} onClick=
{selectAllVisible}>
                Select all
              </button>
            </div>
          )}

          <ul id={listboxId} ref={listRef} role="listbox" aria-multiselectable="true" aria-label=
{label} className="ms-list">
            {filtered.length === 0 ? (
              <li className="ms-empty" role="presentation">
                No results for "{query}"
              </li>
            ) : (
              filtered.map((o, i) => {
                const selected = selectedSet.has(o.id);
                return (
                  <li
                    key={o.id}
                    id={optionDomId(o.id)}
                    data-index={i}
                    role="option"
                    aria-selected={selected}
                    className={`ms-option ${i === activeIndex ? "is-active" : ""}`}
                    onMouseDown={(e) => e.preventDefault()} // keep focus in the input
                    onMouseMove={() => i !== activeIndex && setActiveIndex(i)}
                    onClick={() => toggle(o.id)}
                  >
                    <span className={`ms-check ${selected ? "is-checked" : ""}`} aria-hidden="true"
/>
                    <Highlight text={o.label} query={query} />
                  </li>
                );
              })
            )}
          </ul>
        </div>
```

```
      )}

      <p className="sr-only" aria-live="polite">
        {isOpen ? `${filtered.length} results. ` : ""}
        {value.length} selected.
      </p>
    </div>
  );
}
```

src/components/MultiSelect/MultiSelect.css

```
.ms {
  position: relative;
  width: 100%;
  max-width: 420px;
  font: 14px/1.4 system-ui, sans-serif;
}

.ms-label {
  display: block;
  margin-bottom: 6px;
  font-weight: 600;
}

.ms-control {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 6px;
  min-height: 42px;
  padding: 6px 36px 6px 8px;
  border: 1px solid #c7cbd4;
  border-radius: 8px;
  background: #fff;
  cursor: text;
  position: relative;
}

.ms-control.is-open,
.ms-control:focus-within {
  border-color: #3b5bdb;
  box-shadow: 0 0 0 3px rgb(59 91 219 / 0.2);
}
```

```
.ms-chips {
  display: contents;
  list-style: none;
}

.ms-chip {
  display: inline-flex;
  align-items: center;
  gap: 4px;
  padding: 2px 4px 2px 8px;
  border-radius: 999px;
  background: #e7ecff;
  color: #2b3a8c;
}

.ms-chip-remove,
.ms-clear {
  border: 0;
  background: transparent;
  cursor: pointer;
  font-size: 16px;
  line-height: 1;
  color: inherit;
  border-radius: 50%;
  width: 20px;
  height: 20px;
}

.ms-chip-remove:hover,
.ms-chip-remove:focus-visible {
  background: #c9d3ff;
}

.ms-clear {
  position: absolute;
  right: 8px;
  top: 50%;
  transform: translateY(-50%);
  color: #6b7280;
}

.ms-input {
  flex: 1;
  min-width: 80px;
```

```
  border: 0;
  outline: 0;
  font: inherit;
  padding: 4px 0;
}

.ms-popup {
  position: absolute;
  z-index: 10;
  top: calc(100% + 4px);
  left: 0;
  right: 0;
  border: 1px solid #c7cbd4;
  border-radius: 8px;
  background: #fff;
  box-shadow: 0 8px 24px rgb(0 0 0 / 0.12);
  overflow: hidden;
}

.ms-actions {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 6px 10px;
  border-bottom: 1px solid #eef0f4;
  color: #6b7280;
  font-size: 12px;
}

.ms-actions button {
  border: 0;
  background: none;
  color: #3b5bdb;
  cursor: pointer;
  font: inherit;
  font-weight: 600;
}

.ms-list {
  max-height: 240px;
  overflow-y: auto;
  margin: 0;
  padding: 4px 0;
  list-style: none;
}
```

```
.ms-option {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 8px 10px;
  cursor: pointer;
}

.ms-option.is-active {
  background: #f0f3ff;
}

.ms-option mark {
  background: #ffe58f;
  color: inherit;
  border-radius: 2px;
}

.ms-check {
  width: 16px;
  height: 16px;
  border: 1.5px solid #9aa1b1;
  border-radius: 4px;
  flex-shrink: 0;
}

.ms-check.is-checked {
  background: #3b5bdb;
  border-color: #3b5bdb;
  box-shadow: inset 0 0 0 3px #fff;
}

.ms-empty {
  padding: 12px 10px;
  color: #6b7280;
}

.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  overflow: hidden;
  clip: rect(0 0 0 0);
```

```
  white-space: nowrap;
}
```

src/App.jsx

```
import { useState } from "react";
import MultiSelect from "./components/MultiSelect/MultiSelect";
import { SKILLS } from "./data/skills";

export default function App() {
  const [skills, setSkills] = useState(["react"]);

  return (
    <main className="app">
      <h1>Multi-select dropdown with search</h1>
      <MultiSelect label="Skills" options={SKILLS} value={skills} onChange={setSkills}
placeholder="Search skills..." />
      <pre className="app-output">{JSON.stringify(skills)}</pre>
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

src/index.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
```

```
  font-family: system-ui, sans-serif;
  background: #f6f7fb;
  color: #1f2430;
}

ul {
  margin: 0;
  padding: 0;
}

.app {
  max-width: 480px;
  margin: 48px auto;
  padding: 0 16px;
}

.app-output {
  margin-top: 24px;
  padding: 12px;
  background: #fff;
  border-radius: 8px;
  white-space: pre-wrap;
}
```

Quick self-check of the pure logic (`node selfcheck.mjs`)

```
import assert from "node:assert/strict";
import { filterOptions, toggleValue, splitByMatch } from "./src/utils/options.js";
const opts = [{ id: "js", label: "JavaScript" }, { id: "ts", label: "TypeScript" }, { id: "react",
label: "React" }];
assert.equal(filterOptions(opts, "").length, 3);
assert.equal(filterOptions(opts, "   ").length, 3);
assert.deepEqual(filterOptions(opts, "SCRIPT").map(o => o.id), ["js", "ts"]);
assert.deepEqual(filterOptions(opts, "zzz"), []);
assert.deepEqual(toggleValue(["js"], "ts"), ["js", "ts"]);
assert.deepEqual(toggleValue(["js", "ts"], "js"), ["ts"]);
assert.deepEqual(splitByMatch("JavaScript", "script"), ["Java", "Script", ""]);
assert.deepEqual(splitByMatch("React", ""), ["React", "", ""]);
assert.deepEqual(splitByMatch("React", "x"), ["React", "", ""]);
console.log("selfcheck ok");
```

12. Wrap-up

Edge cases handled

- Empty and whitespace-only queries, no results, unknown ids in `value`.
- No duplicate ids from "Select all"; selection order kept.
- Enter never submits a parent form; Escape only swallowed when it actually closes something.
- Focus never lost while picking; focus returned after chip remove and clear.
- Unique ids for several instances on one page (`useId`).

Trade-offs to mention

Decision	Why	Cost
Controlled `value` (ids)	Parent owns form data; ids survive refetch	Parent must hold state
Focus stays in input + `aria-activedescendant`	User can keep typing; standard combobox pattern	Must manage active index by hand
`pointerdown` on document for outside click	Fires before focus changes, works for touch too	One global listener while open
"Select all" = visible only	Matches what the user sees after filtering	Needs a clear label in a real product
Simple `includes` matching	Predictable, O(n), no library	No typo tolerance
No virtualisation	Simple DOM, fine for hundreds	Slow at 10k+ options

What to add with more time

- Tests with React Testing Library + `user-event`: typing filters, keyboard toggle, click outside closes, Backspace removes chip.
- Async options: debounce the query, `AbortController` to cancel stale requests, loading and error rows.
- Virtualised list (windowing) for thousands of options.
- `maxSelections`, disabled options, grouped options (`role="group"`).
- Portal + collision-aware positioning so the popup isn't clipped inside scroll containers.
- Hidden inputs / `name` prop so it works in a plain `<form>` submit.

13. Counter questions the interviewer may ask

Requirements

Q1. Why keep the dropdown open after a selection?

In a multi-select the user usually picks several items in a row. Closing after each pick forces a reopen + re-search every time. Single-select closes; multi-select stays open and closes on Escape, Tab or outside click.

Q2. Should "Select all" select everything or only the filtered options?

I chose visible only: if I searched "script" and press Select all, I expect the two scripts, not 20 items I can't see. I'd confirm with the interviewer and label it "Select all results" in a real product.

State design

Q3. Why store selected ids instead of the option objects?

Ids are cheap to compare, serialisable for a form or URL, and still valid after options are refetched (new object references). Objects would break `includes` checks and duplicate data the parent already has.

Q4. Why is `value` a prop and not internal state?

The parent needs the selection to submit or validate it. A controlled component has one source of truth, so resetting the form or preloading saved values is just setting the parent's state.

Q5. Why an array and not a `Set` for `value`?

An array keeps the order the user picked (chip order) and is plain JSON. I derive a `Set` with `useMemo` for O(1) `has` checks while rendering options.

Q6. Why not store `filtered` in state?

It's a pure function of `options` and `query`. Storing it means syncing it in effects and risking stale lists. Deriving it with `useMemo` is always correct and only recomputes when its inputs change.

Q7. Why reset `activeIndex` in the change handler and not in a `useEffect` on `query`?

The effect version renders once with a stale index (possibly out of range), then renders again. Setting both in the same handler gives one render with consistent state.

Click outside

Q8. Why `pointerdown` and not `click` for click-outside?

`pointerdown` fires before focus moves and before the target's own click, so the dropdown closes at the right moment. It also covers mouse, touch and pen in one event. `click` can be swallowed if something stops propagation.

Q9. Why not just close on input `blur`?

Blur fires on `mousedown`, before the option's `click`, so the list unmounts and the click never lands. You'd need `setTimeout` hacks or `relatedTarget` checks. `mousedown` preventDefault on options + document pointerdown is cleaner.

Q10. What breaks if the popup is rendered in a portal?

`rootRef.contains(target)` would be false for clicks in the portal, so it would close on every option click. Fix: check both refs (`[rootRef, popupRef].some(r => r.current?.contains(target))`).

Keyboard and accessibility

Q11. Why `aria-activedescendant` instead of moving focus to each option?

Focus stays in the input, so the user can keep typing at any time, and the screen reader still announces the active option. Moving DOM focus would need a roving tabindex and kill typing.

Q12. What does a screen reader announce here?

"Skills, combobox, expanded", then as you arrow: "TypeScript, not selected, 2 of 2". After Enter, the live region says "2 results. 2 selected." Chip buttons read "Remove TypeScript".

Q13. Why `preventDefault` on Enter but only conditionally on Escape?

Enter inside a form would submit it, so we always stop it while open. Escape is only stopped when we actually close something, so if the dropdown is already closed, a parent modal can still use Escape to close itself.

Q14. How do you make sure the active option is visible?

An effect on `activeIndex` calls `scrollIntoView({ block: "nearest" })` on the element with that `data-index`. `nearest` only scrolls if the option is outside the list's viewport and never scrolls the page more than needed.

Implementation steps

Q15. Why does clicking the chip × call `stopPropagation`?

The chip lives inside the control, whose click handler focuses the input and opens the dropdown. Without it, removing a chip would also open the list, which is surprising.

Q16. How do you keep mouse hover and keyboard from showing two active rows?

There is one `activeIndex`. Mouse move over an option sets it, arrow keys change it. Only the option at `activeIndex` gets the active class.

Q17. How would you highlight multiple matches or fuzzy matches?

`splitByMatch` returns before / match / after for the first match. For all matches, split with a case-insensitive regex built from the escaped query. For fuzzy, the matcher returns matched character indexes and you wrap those.

Performance and scaling ("what if")

Q18. What if there are 1 million options?

Don't send them to the client: search on the server with a `limit`. If it must be local, debounce the query, use a precomputed lowercase label index, and virtualise the list so only ~20 rows are in the DOM.

Q19. What if options come from a slow API?

Debounce the query (~250 ms), cancel stale requests with `AbortController` (or ignore out-of-order responses by request id), show a loading row and an error row with retry, and keep a `Map` cache of seen options so chips for selected ids still render.

Q20. What if the user selects 200 items? The chips will fill the screen.

Collapse after N chips: show the first few and a "+197 more" chip that opens the full list, or switch the control to "200 selected" text. Selection state does not change, only the rendering.

Trade-offs ("why not X?")

Q21. Why not use a native `<select multiple>`?

It has no search, poor styling options, and ctrl/cmd-click to multi-select is unknown to most users and awkward on desktop. It's a fine fallback with no JS, but it doesn't meet the search and chips requirements.

Q22. Why not use a library like react-select or Downshift?

In production I'd seriously consider Downshift (headless, accessible) or react-select. In an interview the point is to show I understand the combobox pattern, focus and ARIA, so I build it; I'd mention the libraries as the real-world choice.

Q23. How would you test this?

React Testing Library with `user-event`: type and assert filtered options by role `option`; press ↓ and Enter and assert `aria-selected`; click outside and assert the listbox is gone; Backspace and assert the chip is removed; plus `jest-axe` for accessibility violations. The pure utils get plain unit tests.